

TLS Fingerprinting: How It Works & How to Bypass It (2025) - Browserless

Model

sourceFile: "TLS Fingerprinting: How It Works & How to Bypass It (2025) - Browserless" exportedBy: "Kortex" exportDate: "2026-05-13T11:59:05.584Z"

TLS Fingerprinting: How It Works & How to Bypass It (2025) - Browserless

62995cc6-ca85-49e9-b9dc-b947c89e2699

TLS Fingerprinting: How It Works & How to Bypass It (2025) - Browserless

24cdca43-f1e1-4569-a035-ae2883600674

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer>

TLS Fingerprinting: How It Works & How to Bypass It (2025)

<https://www.browserless.io/>

<https://www.browserless.io/feature/browserql-browser-automation-tool>

Browsers as a Service

<https://www.browserless.io/feature/browsers-as-a-service>

<https://www.browserless.io/feature/rest-apis>

Enterprise Features

<https://www.browserless.io/feature/enterprise-features>

<https://www.browserless.io/scraping>

<https://www.browserless.io/automation>

PDFs & Screenshot

<https://www.browserless.io/pdf-screenshot-screencast>

<https://www.browserless.io/testing>

Bot Detection & Captcha Solving

<https://www.browserless.io/feature/bypass-bot-detection>

Cookies & Reconnects

<https://www.browserless.io/feature/cookies-reconnects>

Hybrid Automations

<https://www.browserless.io/feature/hybrid-automations>

Persisting Sessions

<https://www.browserless.io/feature/persisting-sessions>

Session Replay

<https://www.browserless.io/feature/session-replay>

Live Debugger

<https://www.browserless.io/feature/live-debugger>

Browser Extensions

<https://www.browserless.io/feature/browser-extensions>

Lighthouse Testing

<https://www.browserless.io/feature/lighthouse-performance-api>

Smart Scrape API

<https://www.browserless.io/feature/scrapper-api>

<https://www.browserless.io/feature/mcp-browser-automation>

<https://docs.browserless.io/>

<https://www.browserless.io/pricing>

<https://www.browserless.io/account>

<https://github.com/browserless/browserless>

<https://www.browserless.io/contact>

<https://www.browserless.io/signup/email?plan=free>

Toggle theme

<https://www.browserless.io/account>

<https://www.browserless.io/signup/email?plan=free>

<https://www.browserless.io/contact>

<https://www.browserless.io/feature/browserql-browser-automation-tool>

Browsers as a Service

<https://www.browserless.io/feature/browsers-as-a-service>

<https://www.browserless.io/feature/rest-apis>

Enterprise Features

<https://www.browserless.io/feature/enterprise-features>

<https://www.browserless.io/scraping>

<https://www.browserless.io/automation>

PDFs & Screenshot

<https://www.browserless.io/pdf-screenshot-screencast>

<https://www.browserless.io/testing>

Bot Detection & Captcha Solving

<https://www.browserless.io/feature/bypass-bot-detection>

Cookies & Reconnects

<https://www.browserless.io/feature/cookies-reconnects>

Hybrid Automations

<https://www.browserless.io/feature/hybrid-automations>

Persisting Sessions

<https://www.browserless.io/feature/persisting-sessions>

Session Replay

<https://www.browserless.io/feature/session-replay>

Live Debugger

<https://www.browserless.io/feature/live-debugger>

Browser Extensions

<https://www.browserless.io/feature/browser-extensions>

Lighthouse Testing

<https://www.browserless.io/feature/lighthouse-performance-api>

Smart Scrape API

<https://www.browserless.io/feature/scrapper-api>

<https://www.browserless.io/feature/mcp-browser-automation>

<https://docs.browserless.io/>

<https://www.browserless.io/pricing>

Toggle theme

TLS Fingerprinting: Explanation, Detection and Bypassing it in Playwright and Puppeteer

<https://www.browserless.io/blog/author/alejandro-loyola>

Introduction

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#introduction>

What is TLS?

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#what-is-tls>

What is TLS fingerprinting?

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#what-is-tls-fingerprinting>

How TLS fingerprinting works

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#how-tls-fingerprinting-works>

JA3 TLS fingerprinting

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#ja3-tls-fingerprinting>

JA4 TLS fingerprinting

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#ja4-tls-fingerprinting>

Why it's hard to spoof perfectly

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#why-it-s-hard-to-spoof-perfectly>

Who uses TLS fingerprinting?

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#who-uses-tls-fingerprinting>

Fraud and bot management platforms

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#fraud-and-bot-management-platforms>

CDNs and WAFs

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#cdns-and-wafs>

Enterprise security and SOC tools

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#enterprise-security-and-soc-tools>

Open tooling and research

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#open-tooling-and-research>

How to bypass TLS Fingerprint detection

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#how-to-bypass-tls-fingerprint-detection>

Bypass TLS Fingerprint detection in Playwright – with Browserless

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#bypass-tls-fingerprint-detection-in-playwright-with-browserless>

Bypass Fingerprint detection in Puppeteer – with Browserless

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#bypass-fingerprint-detection-in-puppeteer-with-browserless>

Other ways to bypass TLS Fingerprint detection

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#other-ways-to-bypass-tls-fingerprint-detection>

1. Use TLS-emulating HTTP clients

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#1-use-tls-emulating-http-clients>

2. Use TLS spoofing proxies or middleboxes

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#2-use-tls-spoofing-proxies-or-middleboxes>

3. Randomized or dynamic TLS behavior

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#3-randomized-or-dynamic-tls-behavior>

4. Harden the rest of your fingerprint

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#4-harden-the-rest-of-your-fingerprint>

5. Use BQL when headless stacks hit a wall

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#5-use-bql-when-headless-stacks-hit-a-wall>

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#conclusion>

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#faq>

What is TLS fingerprinting?

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#what-is-tls-fingerprinting>

What is the difference between JA3 and JA4 fingerprinting?

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#what-is-the-difference-between-ja3-and-ja4-fingerprinting>

Who uses TLS fingerprinting?

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#who-uses-tls-fingerprinting>

Why does my Playwright or Puppeteer script get blocked even with good proxies?

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#why-does-my-playwright-or-puppeteer-script-get-blocked-even-with-good-proxies>

How do I bypass TLS fingerprint detection in Playwright?

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#how-do-i-bypass-tls-fingerprint-detection-in-playwright>

How do I bypass TLS fingerprint detection in Puppeteer?

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#how-do-i-bypass-tls-fingerprint-detection-in-puppeteer>

What is BrowserQL (BQL) and when should I use it?

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#what-is-browserql-bql-and-when-should-i-use-it>

Can I completely hide my automation from TLS fingerprinting?

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer#can-i-completely-hide-my-automation-from-tls-fingerprinting>

Introduction

If you automate the modern web, you've probably seen this pattern: Your Playwright or Puppeteer script loads

<https://example.com>

<https://example.com/>

just fine. You point it at a "real" target behind Cloudflare or another WAF. Suddenly you get 403s, infinite redirects, or blank pages – even with "perfect" headers and good proxies.

TLS fingerprinting could be the cause: the inspection of the TLS/SSL handshake itself to identify automation, malware, or other non-standard clients, before any HTML is returned.

In this guide, you'll learn:

What TLS (Transport Layer Security) actually does in the stack.

How TLS fingerprinting works (JA3, JA4, cipher suites, extensions, elliptic curves, and so on).

Who uses TLS fingerprinting and why.

How to legitimately bypass TLS fingerprint detection when you're using Playwright or Puppeteer – especially when you route traffic through Browserless and BrowserQL (BQL).

Other approaches, and when they make sense.

The focus is on reliability and security: techniques you can use for your own apps, your own web servers, or third-party sites where you have permission – not on hiding malicious activity.

What is TLS?

Transport Layer Security (TLS) is the protocol that sits under HTTPS and most modern encrypted protocols. When you see `https://` in the browser, you're looking at HTTP "inside" a TLS tunnel.

At a high level, TLS does three things for a client and server:

Authentication

– proving you're really talking to the domain in the certificate, usually via X.509 certificates and public key crypto.

Confidentiality

– encrypting application data, so observers can't see the content.

- detecting tampering with messages in transit.

A typical TLS 1.2 or TLS 1.3 session looks like this, skipping some details:

ClientHello

- the browser (or other web client) sends a message with:

Supported TLS versions (for example TLS 1.2, 1.3). A list of cipher suites (combinations of key exchange, encryption, and MAC algorithms). Supported elliptic curves for ECDHE. Various extensions (SNI, ALPN, supported groups, etc.).

Supported TLS versions (for example TLS 1.2, 1.3).

A list of cipher suites (combinations of key exchange, encryption, and MAC algorithms).

Supported elliptic curves for ECDHE.

Various extensions (SNI, ALPN, supported groups, etc.).

ServerHello

- the server picks a TLS version, cipher suite and parameters from that list.

Certificate

- the server sends its certificate chain; the client validates it (issuer, hostname, expiry).

Key agreement

- both sides derive shared keys. In modern TLS, this is typically ECDHE over a named curve.

Encrypted application data

- once keys are agreed, all further HTTP exchanges are wrapped in TLS records and encrypted.

For fingerprinting, the important part is that the ClientHello is visible in cleartext on the network. Even though the actual HTTP traffic is encrypted, the way you propose the TLS connection – the version, order of cipher suites, supported extensions, curve preferences, etc. – leaks the unique fingerprint of your TLS stack.

Different TLS libraries – OpenSSL, BoringSSL, SChannel, Go's crypto/tls, Node's tls module – build different ClientHello messages, even for the same https URL. Modern fingerprinting leans hard on those differences.

What is TLS fingerprinting?

When you hear "fingerprinting", your mind probably goes first to actual human fingers and friction ridges. Traditional methods for physical fingerprint detection and fingerprint analysis include powder dusting, cyanoacrylate ("super glue") fuming, vacuum metal deposition and alternate light source techniques.

TLS fingerprinting borrows this metaphor.

Rather than friction ridges, you have:

TLS version and ordering of cipher suites.

Enabled extensions.

Curve and signature preferences.

HTTP/2 and TCP settings layered on top.

Instead of dusting a crime scene, you inspect network flows. Instead of ridge details, you have ordered decimal values representing cipher IDs, extension IDs, and curve IDs.

Instead of a powder-developed image, you end up with a compact, easily consumable resulting string (for example a JA3 or JA4 hash) that lets you identify a TLS stack with a high degree of confidence.

The idea is the same:

Capture a pattern from the client (latent prints or a TLS handshake).

Turn it into a canonical representation.

Compare it against known prints to determine if you've seen the same fingerprint before.

In the web context, "fingerprinting" covers a lot:

Browser fingerprinting

– combining headers, JS features, fonts, canvas, audio, WebGL, and more.

TLS fingerprinting

- looking only at the TLS/SSL handshake.

Device fingerprinting

- combining TLS data with device characteristics from authentication flows, devices, and other telemetry.

TLS fingerprinting is just one band in that larger spectrum, but it has a few special properties that make defenders love it.

How TLS fingerprinting works

Most modern TLS fingerprinting schemes boil the ClientHello (and sometimes related TCP behavior) into a canonical string that acts like a hashable ID.

Two families dominate:

JA3 TLS fingerprinting

JA3 is an older technique, but still widely deployed. It:

Extracts from the ClientHello:

TLS version (for example, 771 for TLS 1.2, 772 for TLS 1.3).
List of cipher suites (as numeric IDs).
List of TLS extensions.
Supported elliptic curves.
EC point formats.

TLS version (for example, 771 for TLS 1.2, 772 for TLS 1.3).

List of cipher suites (as numeric IDs).

List of TLS extensions.

Supported elliptic curves.

EC point formats.

Serializes each list into a separated string of decimal values.

Concatenates these fields with commas.

Hashes that whole thing (traditionally with MD5) into a 32-character hex string – the JA3 hash.

So a JA3 value is a resulting string that collapses a lot of handshake detail into one identifier. Different clients – Chrome, Safari, Firefox, cURL, Go, Node's https – tend to have distinct JA3s.

JA4 TLS fingerprinting

JA4 is a newer family that addresses weaknesses in JA3 – for example, browsers randomizing the order of extensions to break naive matching.

Normalizes (canonicalizes) fields like cipher suites and extensions so randomization doesn't break matching.

Adds more context from the TCP and TLS layers, such as ALPN, SNI behavior, TCP options, and HTTP/2 settings.

Produces a 36-character identifier for a client's TLS configuration.

There are variants:

JA4 – client TLS fingerprint.

JA4S – server TLS response fingerprint.

JA4H – HTTP client fingerprint.

Other JA4+ variants for SSH and additional layers.

Why it's hard to spoof perfectly

Even though JA3/JA4 fingerprints are just strings, they're derived from things that are not trivial to fake at scale:

The full set and order of cipher suites.

Supported curves and signature schemes.

Supported TLS versions.

Extensions and their ordering.

HTTP/2 parameters and TCP options

The "r i" of each packet – the raw record/identifier values – is observable, but the behavior over many connections matters just as much.

Real browsers change these over time as versions ship. Fingerprinting databases get updated. If your automation uses a static TLS stack or a generic TLS library that doesn't match any common browser, a web server or WAF can detect that very early in the TLS handshake, before any meaningful analysis of application data.

Who uses TLS fingerprinting?

TLS fingerprinting has become a standard building block in security and anti-bot pipelines, especially on the server side.

Fraud and bot management platforms

Fraud and bot detection vendors combine JA3/JA4 with cookies, IP reputation, and behavioral features to spot credential stuffing, carding, and scraping.

Examples include: Fingerprint, which offers commercial device and TLS fingerprinting APIs that combine TLS fingerprints with other browser and device signals, as well as other risk-scoring services that advertise TLS fingerprint support for high-risk login and payment flows.

CDNs and WAFs

Content delivery networks increasingly use TLS fingerprints in their "is this a real browser?" logic.

uses JA4-style fingerprints and other "inter-request signals" to improve threat detection at the edge.

discusses TLS fingerprinting as a key part of modern security defenses in their research on what's working and what isn't.

These systems compare incoming fingerprints with a database of "normal" patterns and flag anomalies.

Enterprise security and SOC tools

Security teams use TLS fingerprints for:

Threat hunting – tracking malware families that reuse the same TLS stack.

Detections – spotting headless automation or old libraries in regulated environments.

Policy enforcement – blocking deprecated TLS versions or weak cipher configurations.

Google's security ecosystem, for example, integrates JA4 fingerprints into its threat intelligence to distinguish different client stacks with high precision.

Open tooling and research

There's also an ecosystem of open tooling around TLS fingerprinting, including libraries implementing JA3/JA4 encoders/decoders, testing sites that show your TLS profile, and research projects analyzing differences between headless and headed browsers, including TLS behavior.

All of this means TLS fingerprinting is a mainstream technique, not a niche curiosity.

How to bypass TLS Fingerprint detection

To bypass TLS fingerprint detection in a legitimate way:

Use real browsers (Chromium, Firefox, WebKit) instead of raw TLS libraries.

Align TLS profiles (TLS version, cipher suites, extensions, elliptic curves) with popular browsers.

Avoid mismatched stacks by not sending Chrome-like headers over a non-Chrome TLS library.

Offload TLS details to a managed platform (such as Browserless +

<https://www.browserless.io/feature/browserql-browser-automation-tool>

) that already maintains realistic TLS fingerprints.

Test your automation against TLS fingerprint test sites and your own logs, and update regularly.

You're not becoming invisible; you're reducing obvious gaps between your automation and what a normal user's browser looks like.

Now let's look at how that plays out in Playwright and Puppeteer, focusing on Browserless as the "production-grade" option instead of hand-rolling TLS logic everywhere.

Bypass TLS Fingerprint detection in Playwright – with Browserless

When you run Playwright locally with `chromium.launch()`, the TLS handshake comes from the browser binary itself (Chromium, Chrome, etc.), not Node's TLS library. That already looks much more like a real user than raw HTTP clients.

The gaps usually appear when you:

Use complex proxy chains or MITM proxies that terminate TLS with a non-browser TLS library.

Run unusual Chrome flags or custom builds.

Mix non-browser HTTP clients (like `fetch` via Node) with browser traffic and expect them to look the same.

Browserless shifts most of that complexity away from your code: You connect Playwright to Browserless over WebSocket (CDP), and Browserless runs real browsers in its own infrastructure and proxies your commands into them.

Launch options

<https://docs.browserless.io/baas/launch-options>

(including stealth settings) are expressed as query parameters or a JSON launch payload, and Browserless takes care of aligning TLS, HTTP/2, and other low-level details with real, modern browser profiles.

In practice, a typical Playwright + Browserless flow looks like this (TypeScript/Node):

```
import { chromium } from 'playwright-core';

const TOKEN = process.env.BROWSERLESS_TOKEN!;

async function run() { // Browserless launch options – note `stealth` is handled by
  Browserless const launchOptions = { headless: true, stealth: true, // enable Browserless
    anti-detection profile args: ['--window-size=1920,1080'], };

  const queryParams = new URLSearchParams({ token: TOKEN, launch:
    JSON.stringify(launchOptions), });

  const wsEndpoint = `wss://production-sfo.browserless.io?${queryParams.toString()}`;

  // Connect over Chrome DevTools Protocol – TLS to the target site is handled by
  Browserless const browser = await chromium.connectOverCDP(wsEndpoint);
```



```
const context = await browser.newContext(); const page = await context.newPage();

await page.goto('https://example.com', { waitUntil: 'networkidle' });

// ...your scraping or testing logic here...

await browser.close(); }

run().catch(err => { console.error(err); process.exit(1); });
```

The key advantages of using Browserless to bypass TLS fingerprinting are:

Your code never touches TLS parameters directly – no manual cipher suite lists or TLS version flags.

Browserless uses hardened launch profiles and stealth settings to reduce browser and TLS fingerprints, including JA3/JA4, header ordering, HTTP/2 settings, and more.

You can still configure proxies, session lifetimes, and other behavior at the Browserless layer, but the low-level handshake stays consistent with real browsers.

With Browserless, you can

bypass other automation detection tools, such as those used by Cloudflare

<https://www.browserless.io/blog/how-to-bypass-cloudflare-scraping>

If you hit a particularly aggressive WAF, you can take one more step and offload the page interaction entirely to BrowserQL (BQL), a GraphQL-style API that runs directly on Browserless' infrastructure. It's designed as a stealth-first automation layer that already considers TLS fingerprinting, browser fingerprinting, and behavioral detection.

A minimal BQL call from Node might look like this:

```
import fetch from 'node-fetch';

const TOKEN = process.env.BQL_TOKEN!; const BQL_ENDPOINT = 'https://production-
sfo.browserless.io/stealth/bql';

const query = `mutation VisitAndExtract { goto(url: "https://example.com", waitUntil:
networkIdle) { status url } content: text(selector: "h1") { text } } `;
```

```
async function run() { const res = await fetch(BQL_ENDPOINT, { method: 'POST', headers:
{ 'Content-Type': 'application/json', Authorization: `Bearer ${TOKEN}` }, }, body:
JSON.stringify({ query }), });

const body = await res.json(); console.log(body.data.content.text); }

run().catch(console.error);
```

In this example, TLS from your code to Browserless is just standard https to a trusted API you control. TLS from Browserless to the target site uses Browserless' own tuned profiles and stealth routes, including TLS and browser fingerprint mitigations.

Bypass Fingerprint detection in Puppeteer – with Browserless

You can use Browserless to bypass fingerprint detection in Puppeteer in a similar way. A local `puppeteer.launch()` gives you Chrome with a realistic TLS profile, but you're responsible for infrastructure, proxies, and staying up-to-date.

Connecting to Browserless via `puppeteer.connect()` lets you reuse your existing Puppeteer scripts while delegating TLS and stealth behavior to Browserless. Read our

ultimate guide to Puppeteer web scraping

<https://www.browserless.io/blog/ultimate-guide-to-puppeteer-web-scraping>

for more tips.

Here's a simplified example using puppeteer-core:

```
import puppeteer from 'puppeteer-core';

const TOKEN = process.env.BROWSERLESS_TOKEN!;

async function run() { // Launch options Browserless will apply when it starts the
browser const launchOptions = { headless: true, stealth: true, args: ['--window-
size=1920,1080'], };

const params = new URLSearchParams({ token: TOKEN, launch:
JSON.stringify(launchOptions), });

const browser = await puppeteer.connect({ browserWSEndpoint: `wss://production-
sfo.browserless.io?${params.toString()}`, });
```

```
const page = await browser.newPage();

await page.goto('https://example.com', { waitUntil: 'networkidle2' });

const title = await page.title(); console.log('Page title:', title);

await browser.close(); }

run().catch(err => { console.error(err); process.exit(1); });
```

Again, for TLS fingerprinting reasons, Puppeteer speaks Chrome DevTools Protocol over WebSocket to Browserless.

Browserless owns the actual TLS session to the target, and your local environment, Node version, or corporate proxy stack doesn't leak a weird non-browser TLS fingerprint. Stealth settings and launch profiles are also maintained by Browserless, which tunes them to keep up with bot detectors, TLS changes, and JA4+ updates.

This is usually a better long-term strategy than trying to manually bolt TLS spoofing libraries onto your own proxy infrastructure – you get the same kind of control, but someone else maintains the alignment with real browsers for you.

Other ways to bypass TLS Fingerprint detection

If you're not using Browserless yet, or you want to understand the lower layers anyway, there are other techniques you'll see in the wild. Use these carefully and within legal/ToS boundaries.

1. Use TLS-emulating HTTP clients

Some HTTP clients and libraries are explicitly designed to mimic real browsers, including TLS libraries like `tls-client`, which ship predefined browser profiles that match the TLS and HTTP/2 fingerprints of specific Chrome, Firefox, and mobile versions. They let you pick a profile (for example, "Chrome 124 on Windows 10"), and the library adjusts TLS version, cipher suites, extensions, elliptic curves, and HTTP/2 settings accordingly.

In a request-based scraper (no full browser), this can be a big step up from the default `ssl` or `https` modules, which often expose very library-specific fingerprints.

However, there are a few trade-offs:

You must keep profiles updated as real browsers change.

You still need to handle browser-level fingerprinting (JS, canvas, fonts) separately.

You're trusting a third-party implementation for security-sensitive code.

2. Use TLS spoofing proxies or middleboxes

Another pattern is to put a

TLS-aware proxy

in front of your automation: The proxy terminates TLS with a browser-like profile (matching real Chrome, etc.) and your scraper talks http or generic TLS to the proxy.

Some open projects explicitly advertise TLS fingerprint spoofing for privacy or anti-tracking reasons, which gives you a non-contact way to tune TLS – your Playwright/Puppeteer scripts don't change, but the proxy adjusts the TLS handshake for every outgoing connection.

Again, there are a few trade-offs:

You're potentially giving a third party visibility into all your traffic.

Poorly implemented spoofing can create new anomalies (for example, Chrome-like TLS with non-Chrome header sets).

Many providers prohibit scraping-style usage; violating ToS can get your access revoked.

3. Randomized or dynamic TLS behavior

Some advanced evasive setups use dynamic TLS libraries that randomize certain parameters, or switch between realistic profiles over time. For example, rotating between several realistic JA4 fingerprints for different clients, or slightly varying extension order, padding, or HTTP/2 settings to avoid building a stable fingerprint.

The main trade-off is that defenders are catching up with this approach:

Machine-learning models can spot TLS patterns that are "too random" or inconsistent with other signals.

Some WAFs treat high entropy in TLS and browser fingerprints as a risk signal in itself.

As with physical latent prints, over-processing can damage the underlying evidence; too much variation can look unnatural.

4. Harden the rest of your fingerprint

Even if you get TLS right, you can still be flagged by:

Headless-only Chrome flags.

`navigator.webdriver`

and other automation markers.

User-Agent / Client Hints

Inconsistent image formats, canvas output, or font lists.

Tools such as Fingerprint-style browser fingerprint generators (and Browserless' own stealth tooling) aim to keep TLS, headers, DOM state, and JS behavior aligned with real browsers, instead of treating TLS in isolation.

In the same way forensic scientists compare full ridge details and not just one part of a fingerprint, modern bot detectors look across TLS, browser, and behavior. You need to treat the whole stack as the "surface" you're leaving your marks on.

5. Use BQL when headless stacks hit a wall

Playwright and Puppeteer are excellent, but they weren't designed for stealth first; they're testing tools.

Browserless built BQL

<https://docs.browserless.io/browserql/start>

specifically to control the browser directly via Chrome DevTools Protocol, minimize browser and TLS fingerprints out of the box and provide dedicated stealth routes that bundle TLS tuning, session management, proxy rotation, and challenge solving.

When you hit a target that combines TLS fingerprinting, JS challenges, CAPTCHAs, and strict behavior analysis, falling back to BQL can be simpler than endlessly tweaking flags and external libraries.

Use it like any other API: send queries, store results, and keep your Playwright/Puppeteer code for flows where full browser control still makes sense.

CDNs, anti-fraud teams, and bot managers use the TLS handshake – TLS version, cipher suites, extensions, elliptic curves, HTTP/2 settings – to build a unique fingerprint of your TLS stack.

automating with Playwright

<https://www.browserless.io/blog/playwright-heroku>

or Puppeteer, there are three pragmatic takeaways:

Don't fight TLS with low-level hacks unless you must – Prefer real browsers and well-maintained profiles over hand-crafted JA3/JA4 hacks.

Keep the whole fingerprint coherent – TLS, headers, DOM, and behavior all need to look like they come from the same browser, not a patchwork of spoofing tricks.

Lean on managed platforms when stakes are high – Browserless and BQL are effectively "the thing you'd build yourself" – tuned TLS fingerprints, stealth routes, session and proxy management – without you having to babysit cipher lists and extension order.

Used ethically, these techniques help you make your legitimate automation more reliable, test your own defenses against realistic adversaries, understand where TLS fingerprinting fits into the broader network processing and detection pipeline, and improve your web scraping capabilities.

You're never completely undetectable, just harder to distinguish from real users – but that's usually exactly what you need.

What is TLS fingerprinting?

TLS fingerprinting is a technique that inspects the TLS/SSL handshake to identify clients based on their TLS version, cipher suites, extensions, and elliptic curve preferences. This data is compiled into a unique identifier (like a JA3 or JA4 hash) that can distinguish real browsers from automation tools, bots, or malware.

What is the difference between JA3 and JA4 fingerprinting?

JA3 is an older fingerprinting method that extracts TLS version, cipher suites, extensions, elliptic curves, and EC point formats from the ClientHello, then hashes them into a 32-character MD5 string. JA4 is newer and addresses JA3's weaknesses by normalizing fields to handle browser randomization, adding TCP and HTTP/2 context, and producing a 36-character identifier that's harder to evade.

Who uses TLS fingerprinting?

TLS fingerprinting is used by CDNs and WAFs (like Cloudflare and Fastly), fraud and bot detection platforms, enterprise security teams for threat hunting and malware detection, and various anti-bot services that need to distinguish real browsers from automated clients.

Why does my Playwright or Puppeteer script get blocked even with good proxies?

Even with proper headers and proxies, your script may be blocked because the TLS handshake itself reveals you're not using a standard browser. Automation tools, proxy chains, or custom HTTP clients often have TLS fingerprints that don't match real browsers, triggering WAF blocks before any HTML is returned.

How do I bypass TLS fingerprint detection in Playwright?

The most reliable method is to connect Playwright to a managed platform like Browserless via WebSocket (CDP). Browserless runs real browsers with properly tuned TLS profiles, handling cipher suites, extensions, and HTTP/2 settings automatically. This eliminates the need to manually configure TLS parameters in your code.

How do I bypass TLS fingerprint detection in Puppeteer?

Use `puppeteer.connect()` to connect to Browserless instead of launching a local browser with `puppeteer.launch()`. Browserless manages the actual TLS session to target sites using browser-accurate fingerprints, so your local environment doesn't leak non-browser TLS characteristics.

What is BrowserQL (BQL) and when should I use it?

BrowserQL is a GraphQL-style API built by Browserless specifically for stealth-first automation. It handles TLS fingerprinting, browser fingerprinting, and behavioral detection out of the box. Use BQL when you're hitting aggressive WAFs or bot detection systems that combine multiple detection methods beyond just TLS fingerprinting.

Can I completely hide my automation from TLS fingerprinting?

No, you can never be completely undetectable. The goal is to reduce obvious gaps between your automation and real browser traffic. This means aligning TLS profiles, headers, DOM state, and behavior so they all appear to come from the same legitimate browser—not a patchwork of spoofing techniques.

<https://www.browserless.io/>

<https://github.com/browserless/chrome>

<https://twitter.com/browserless>

<https://www.youtube.com/@browserless>

<https://www.browserless.io/scraping>

<https://www.browserless.io/automation>

PDF / Screenshot

<https://www.browserless.io/pdf-screenshot-screencast>

<https://www.browserless.io/testing>

Documentation

<https://docs.browserless.io/>

AI Integrations

<https://docs.browserless.io/ai-integrations/start>

Status page

<https://status.browserless.io/>

<https://www.browserless.io/changelog>

<https://www.browserless.io/about>

<https://www.browserless.io/blog>

<https://www.browserless.io/contact>

© 2026 Browserless

Trust Center

<https://www.browserless.io/trust>

Privacy Policy

<https://www.browserless.io/privacy-policy>

Terms of Service

<https://www.browserless.io/terms-of-service>

Cookie Settings

We use cookies to enhance your experience and analyze site traffic.

<https://www.browserless.io/privacy>

Cookie Settings Decline Accept

Cookie Preferences

Manage your cookie preferences below. Necessary cookies cannot be disabled as they are required for the site to function.

Required for the site to function.

Help us understand how visitors use the site.

Used to deliver relevant ads and campaigns.

Decline All Save Preferences